

Arhitectura calculatoarelor

Curs 7

7. Procesorul cu cicluri multiple de execuție – controlul

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

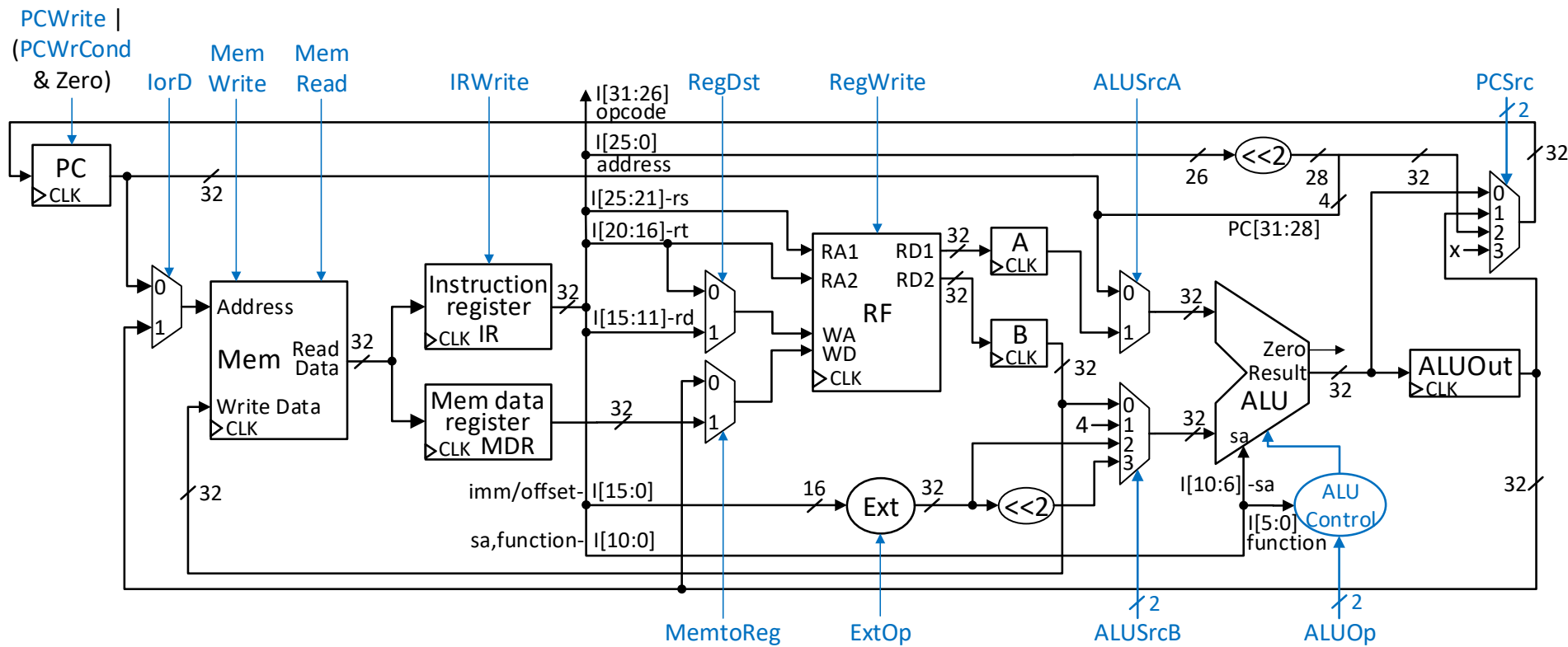
Obiective

1. Proiectarea multi-ciclu
2. Implementarea unității de control
 - 2.1 Implementarea cablată
 - 2.2 Implementarea micro-programată
3. Mecanismul de tratare a excepțiilor

7.1 Proiectarea multi-ciclu

Pași de proiectare – recapitulare

1. Analiza la nivel RTL (pornind de la ISA).
2. Stabilirea componentelor pentru calea de date.
3. Asamblarea componentelor (cu ajutorul RTL).



7.1 Proiectarea multi-ciclu

4. Descrierea RTL concret (pe baza RTL abstract și a căii de date).

Pas	Instrucțiuni de tip R	Instr. aritmetice și logice cu imediat	Instrucțiuni de acces la memorie	Instr. de branch	Instr. de jump
T ₀ : Instruction Fetch	IR ← M[PC], PC ← PC + 4 (se realizează simultan)				
T ₁ : Instruction Decode, OperandFetch	A ← RF[rs], B ← RF[rt], ALUOut ← PC + S_Ext(offset)<<2 (se realizează simultan)				
T ₂ : Execution, address computation, branch / jump completion	ALUOut ← A op B	ORI: ALUOut ← A Z_Ext(imm) ADDI: ALUOut ← A + S_Ext(imm)	ALUOut ← A + S_Ext(offset)	Zero → PC ← ALUOut	PC ← PC[31:28] address 00
T ₃ : Memory access, R-type completion	RF[rd] ← ALUOut	RF[rt] ← ALUOut	LW: MDR ← M[ALUOut] SW: M[ALUOut] ← B		
T ₄ : Memory read completion			LW: RF[rt] ← MDR		

Obs: Registrele MDR, A, B, ALUOut, se scriu la fiecare pas, dar se folosesc numai când au date relevante. IR se scrie în T₀ și apoi se blochează pentru a oferi câmpurile instrucțiunii.

7.1 Proiectarea multi-ciclu

4. Descrierea RTL concret – semnale de control.

Pas	IorD	Mem Read	Mem Write	IR Write	Reg Dst	Mem toReg	Reg Write	Ext Op	ALU SrcA	ALU SrcB _{1:0}	ALUOp _{1:0}	PCSrc _{1:0}	PCWr Cond	PC Write	Fază execuție
T ₀	0	1	0	1	x	x	0	x	0	01	00 (+)	00	x	1	IF
T ₁	x	0	0	0	x	x	0	1	0	11	00 (+)	xx	0	0	ID, OF
T ₂	x	0	0	0	x	x	0	x	1	00	10 (R)	xx	0	0	EX – Tip R
T ₃	x	0	0	0	1	0	1	x	x	xx	xx	xx	0	0	WB – Tip R
T ₂	x	0	0	0	x	x	0	0/1	1	10	11 _() /00 ₍₊₎	xx	0	0	EX
T ₃	x	0	0	0	0	0	1	x	x	xx	xx	xx	0	0	WB
T ₂	x	0	0	0	x	x	0	1	1	10	00 (+)	xx	0	0	EX – LW
T ₃	1	1	0	0	x	x	0	x	x	xx	xx	xx	0	0	M – LW
T ₄	x	0	0	0	0	1	1	x	x	xx	xx	xx	0	0	WB – LW
T ₂	x	0	0	0	x	x	0	1	1	10	00 (+)	xx	0	0	EX – SW
T ₃	1	0	1	0	x	x	0	x	x	xx	xx	xx	0	0	M – SW
T ₂	x	0	0	0	x	x	0	x	1	00	01 (-)	01	1	0	EX – BEQ
T ₂	x	0	0	0	x	x	0	x	x	xx	xx	10	x	1	EX – J

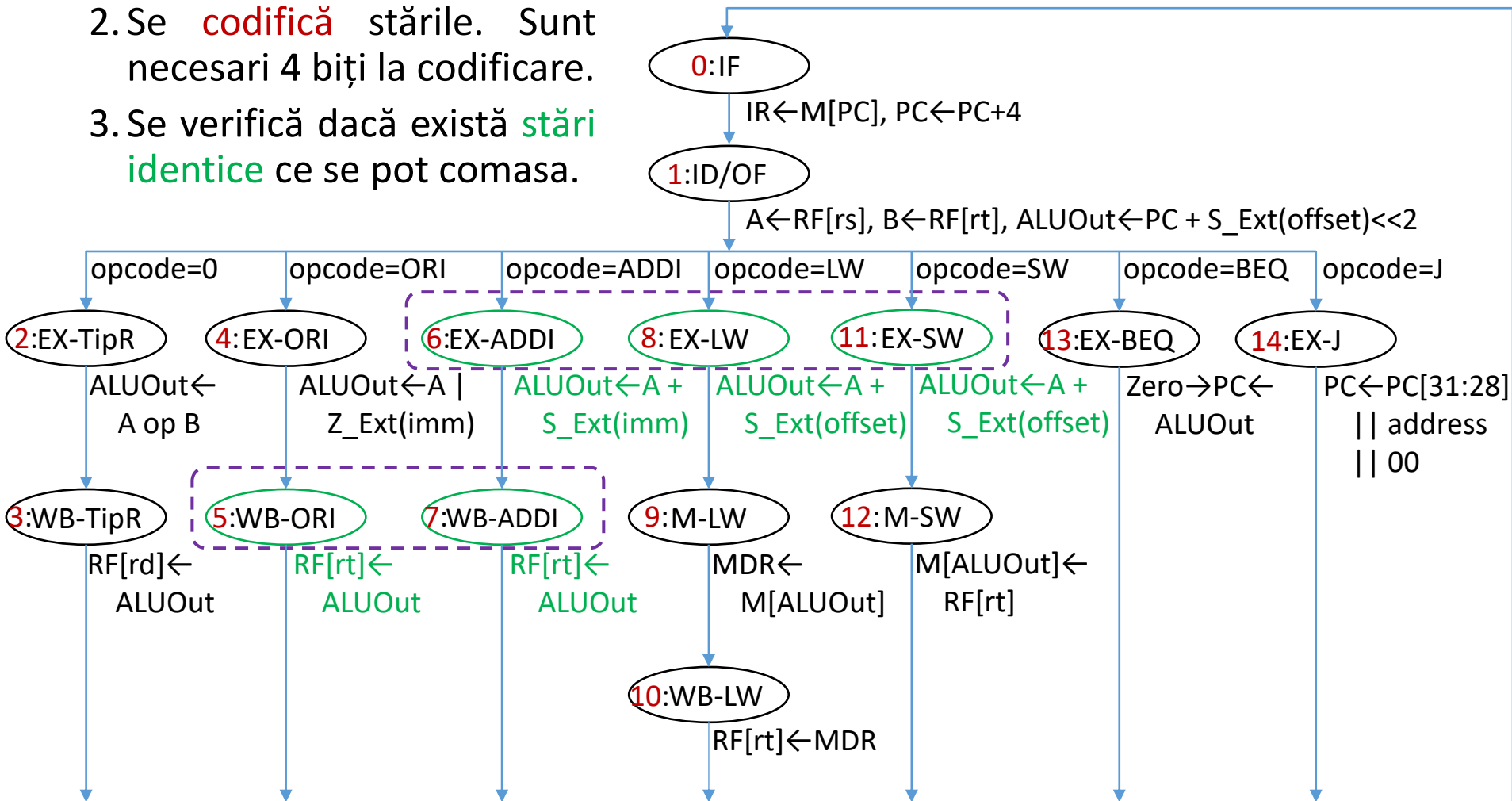
7.1 Proiectarea multi-ciclu

5. Implementarea logicii de control.

1. Se definește FSM.

2. Se **codifică** stările. Sunt necesari 4 biți la codificare.

3. Se verifică dacă există **stări identice** ce se pot comasa.

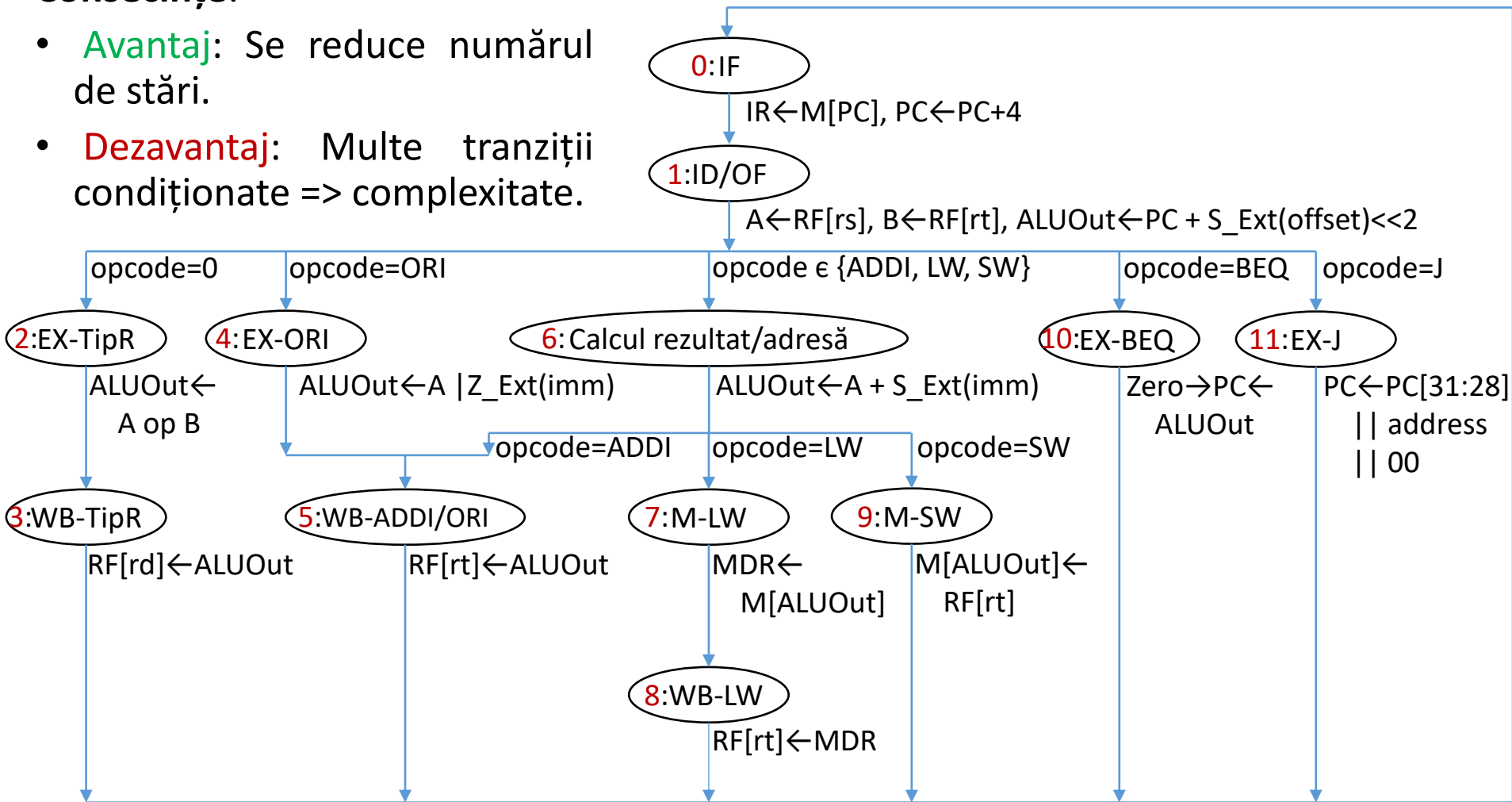


7.1 Proiectarea multi-ciclu

5. Implementarea logicii de control – FSM după comasare

Consecințe:

- **Avantaj:** Se reduce numărul de stări.
- **Dezavantaj:** Multe tranziții condiționate => complexitate.



Notă: În continuare se va implementa numai FSM fără comasări!

7.2 Implementarea unității de control

Variante de implementare

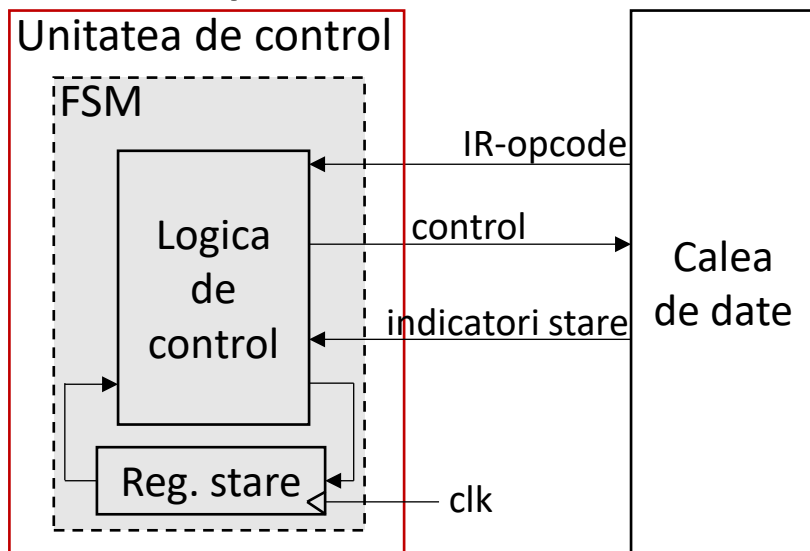
1. Cablată

- viteză de execuție mare;
- complexitate ridicată odată cu creșterea numărului de semnale;
- nu permite reparații/ajustări de către utilizator.

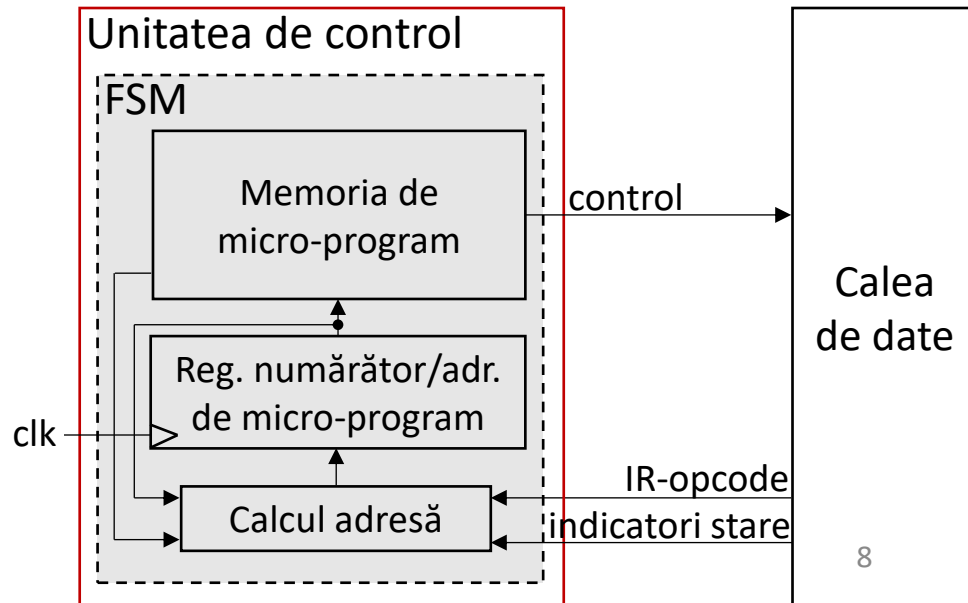
2. Micro-programată

- stările sunt stocate în memorie => mai lentă din cauza accesului la memorie;
- implementare simplă chiar și la unități de control complexe;
- facilitează modificări la nivel de micro-cod.

Implementare cablată



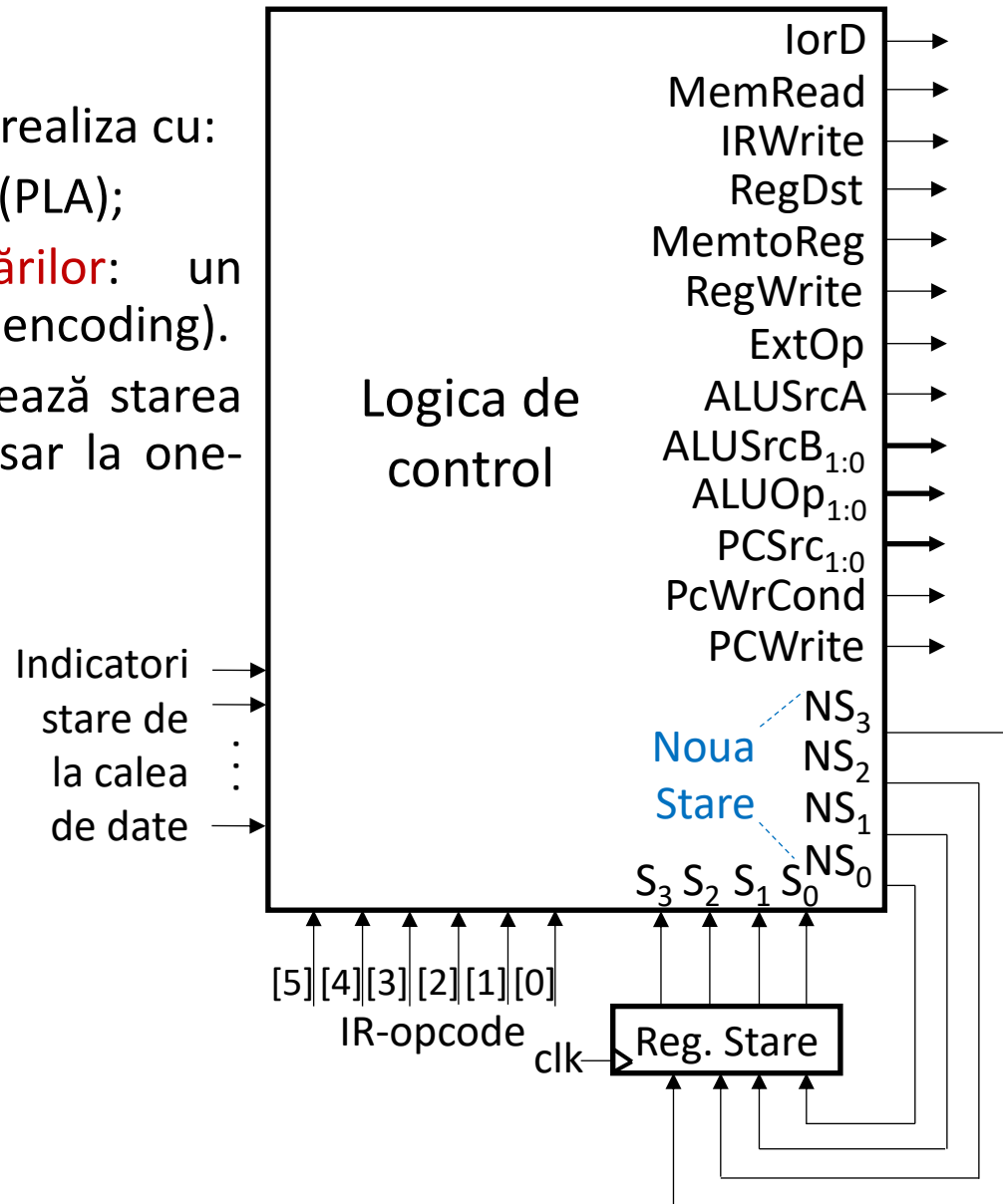
Implementare micro-programată



7.2.1 Implementarea cablată

7.2.1.1 Componente

- Logica de control – se poate realiza cu:
 - Programable Logic Array** (PLA);
 - Bistabile asociate stărilor**: un bistabil pe stare (one-hot encoding).
- Registrul de stare – memorează starea curentă. **Obs**: nu este necesar la one-hot encoding.



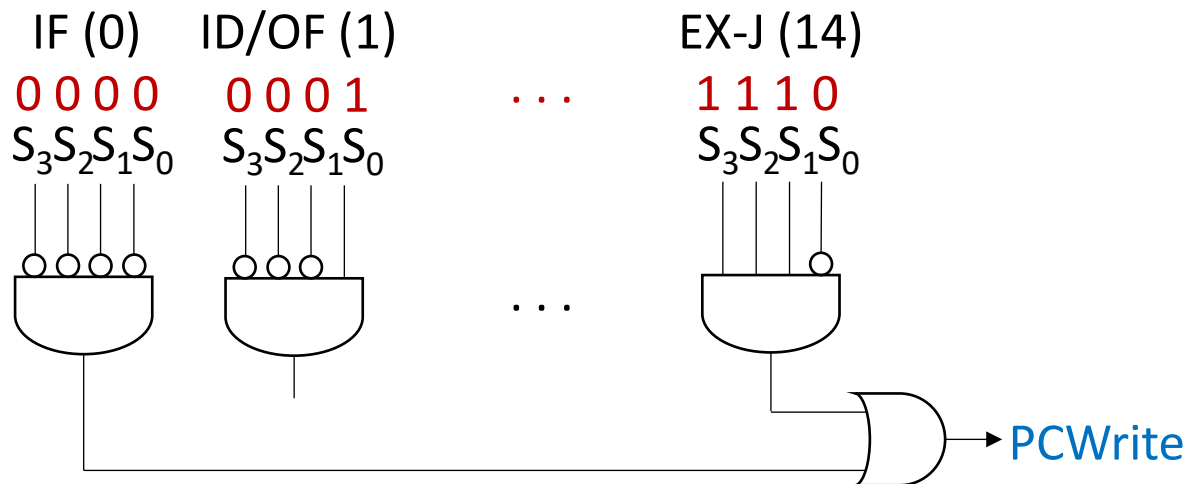
7.2.1 Implementarea cablată

7.2.1.2 Implementarea cu PLA

- Nivel de porți ȘI urmate de un nivel de porți SAU.
- Semnalele de control sunt activate în funcție de starea curentă.
- Biții pentru noua stare $NS_{3:0}$ sunt activați în funcție de starea curentă, câmpul *opcode* și indicatorii de stare de la calea de date.

Exemplu de implementare a unui semnal de control pentru calea de date:

- **PCWrite** se activează la pasul IF (cod=0000₂) și la pasul EX-J (cod=1110₂) conform tabelului și FSM fără comasări.



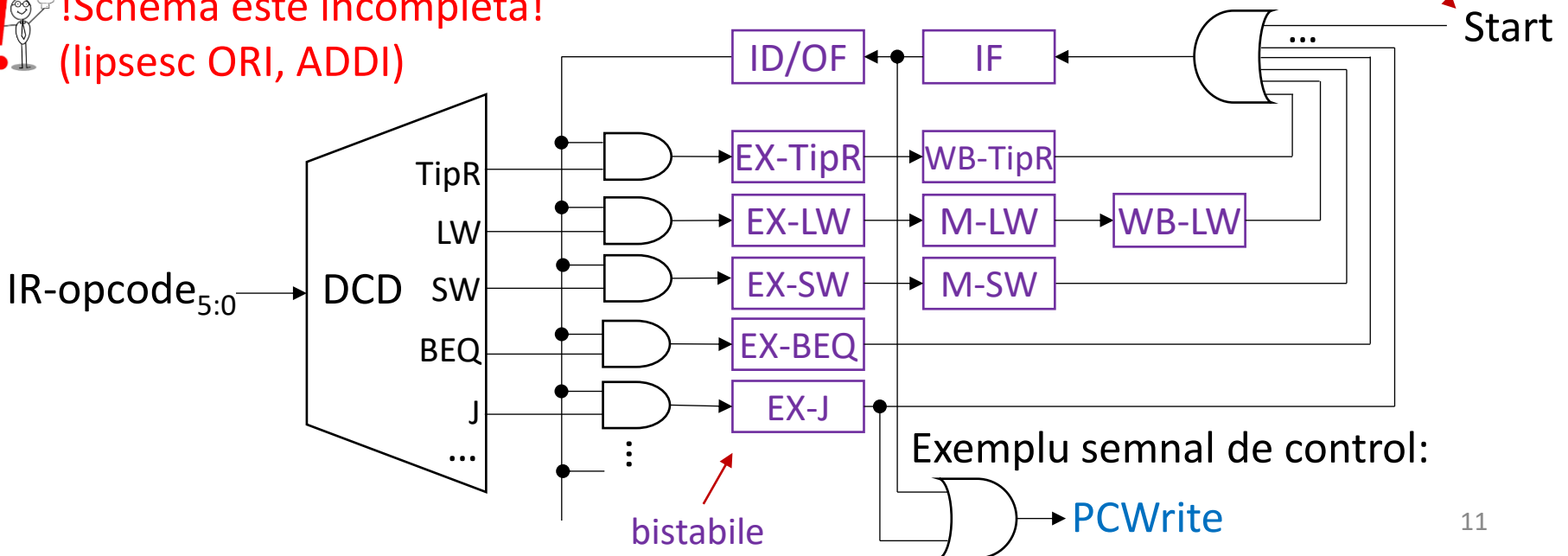
7.2.1 Implementarea cablată

7.2.1.3 Implementarea cu un bistabil pe stare – one-hot encoding

- Se renunță la coduri și se asociază **un bistabil** la fiecare stare. Bistabilul va activa toate semnalele de control care trebuie să fie 1 în starea respectivă.
- ieșirea fiecărui bistabil se leagă la intrarea bistabilului asociat pasului următor în execuție.
- Unificarea mai multor ramificații în fluxul de execuție se face cu poarta SAU.
- **Avantaj:** ușor de extins și de modificat. Facilitează crearea de bucle și ramificații (urmează ex. în continuare).

Semnal care pornește execuția (activează IF): pus pe 1 la inițializare pentru o perioadă de ceas.

!Schema este incompletă!
(lipsește ORI, ADDI)



7.2.1 Implementarea cablată

7.2.1.4 Studiu de caz: Implementarea SLL multi-ciclu

- RTL abstract: $RF[rd] \leftarrow RF[rt] \ll sa$
- Presupunem că ALU face deplasare cu 1 sau 0 biți (sa_{ALU} pe 1 bit) $\Rightarrow$ EX-TipR(T_2) se va repeta un număr variabil de cicluri de ceas până se fac toate deplasările:

- RTL abstract: $RF[rd] \leftarrow RF[rt] \ll sa$
- RTL concret:

ID/OF $T_1: B \leftarrow RF[rt]$

EX { $T_2_{sa}: ALUOut \leftarrow B \ll 1$
 $T_2_{sa-1}: ALUOut \leftarrow ALUOut \ll 1$
 $T_2_{sa-2}: ALUOut \leftarrow ALUOut \ll 1$
...
 $T_2_1: ALUOut \leftarrow ALUOut \ll 1$
 $T_2_0: ALUOut \leftarrow ALUOut \ll 0$

WB $T_3: RF[rd] \leftarrow ALUOut$

Exemplu: $RF[rd] \leftarrow RF[rt] \ll 7$ ($sa=7$)

Pași:

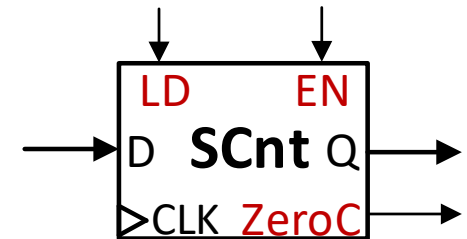
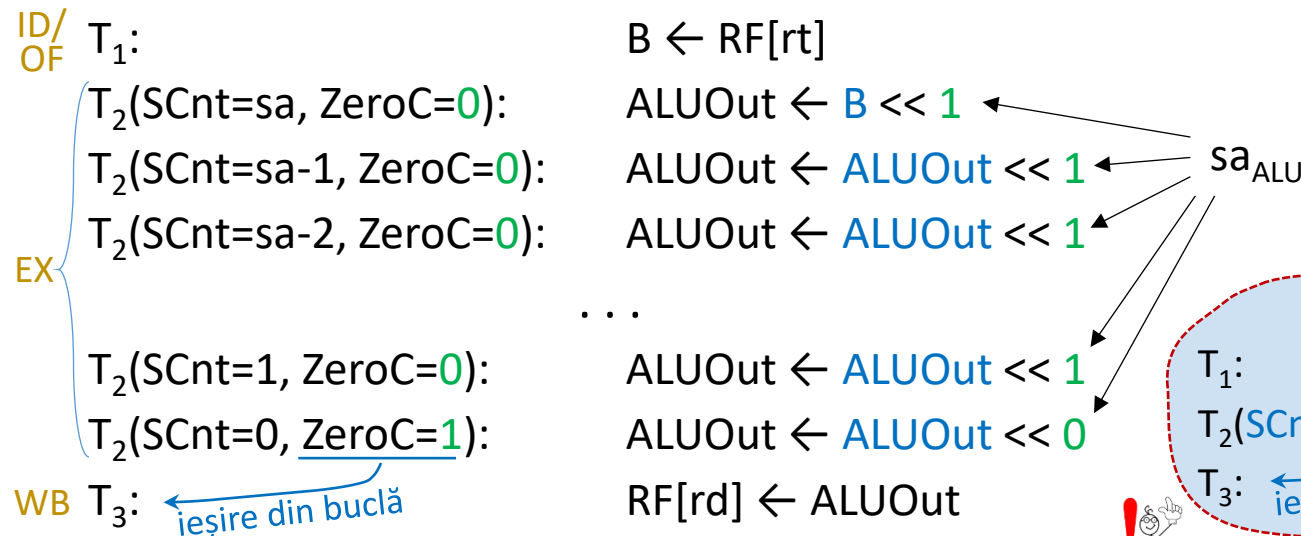
$T_1: B \leftarrow RF[rt]$
 $T_2_7: ALUOut \leftarrow B \ll 1$
 $T_2_6: ALUOut \leftarrow ALUOut \ll 1$
 $T_2_5: ALUOut \leftarrow ALUOut \ll 1$
...
 $T_2_1: ALUOut \leftarrow ALUOut \ll 1$
 $T_2_0: ALUOut \leftarrow ALUOut \ll 0$
 $T_3: RF[rd] \leftarrow ALUOut$

7.2.1 Implementarea cablată

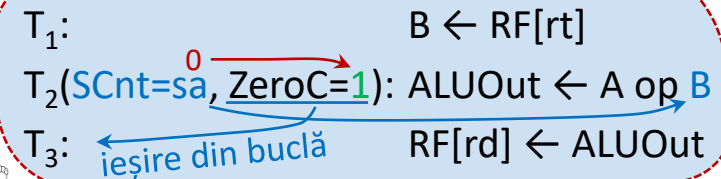
7.2.1.4 Studiu de caz: Implementarea SLL multi-ciclu (cont)

- **Soluție:** Se extinde calea de date cu numărătorul **SCnt** (5 biți) având proprietățile:
 - Comanda **LD** – încărcare sincronă și prioritară față de numărare.
 - Comanda **EN** – numărare descrescătoare. Presupunem că se oprește la 0.
 - Ieșirea **ZeroC** (1 bit) indică atingerea valorii 0.

• RTL concret:



La alte instrucțiuni R



Important: Câmpul **sa** din instrucțiune trebuie completat cu 0 la toate instrucțiunile de tip R, care nu necesită repetarea pasului T₂ de mai multe ori: ex: ADD, SLT, OR, AND, etc.

Obs:

1. Necesari **MUX** pe intrarea inferioară ALU care selectează:
 - **B** – dacă SCnt=sa
 - **ALUOut** – în rest
2. $\text{sa}_{\text{ALU}} \leftarrow \overline{\text{ZeroC}}$
3. ZeroC=1 marchează ieșirea din bucla T₂

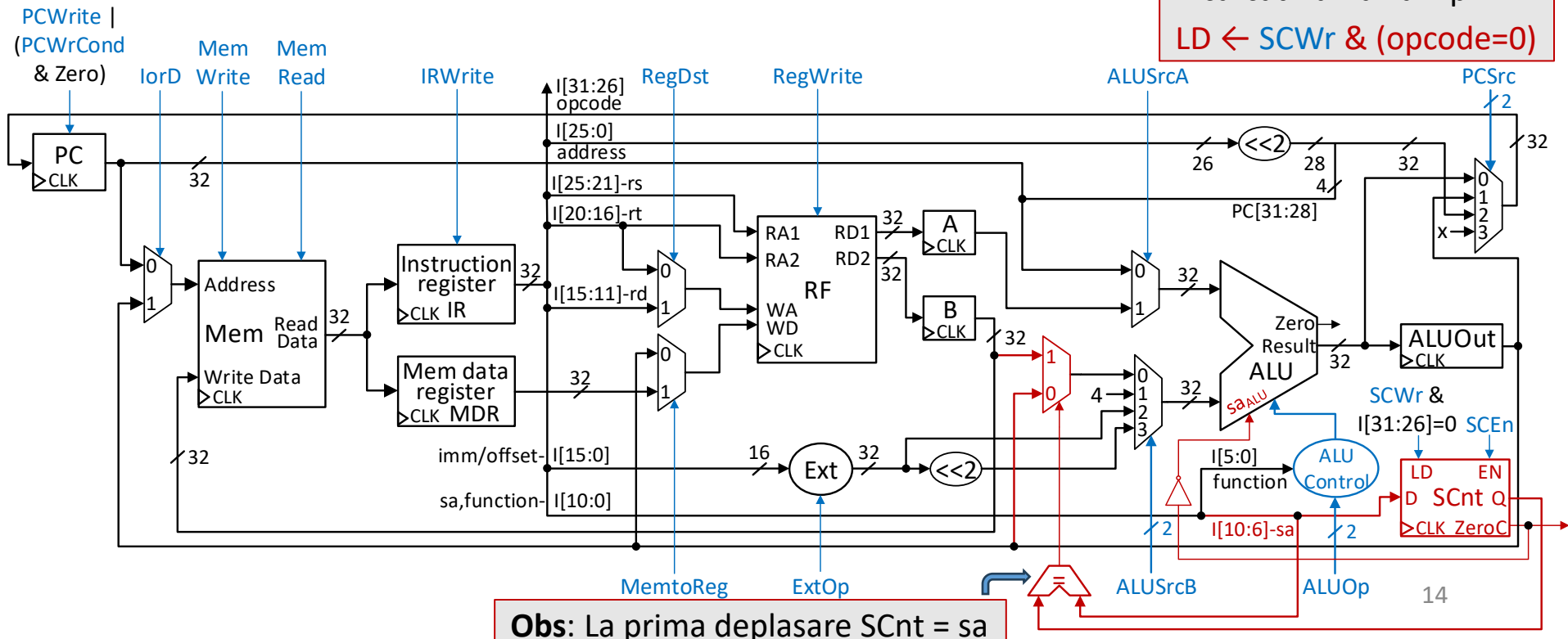
7.2.1 Implementarea cablată

7.2.1.4 Studiu de caz: Implementarea SLL multi-ciclu (cont)

• Soluție:

- Se extinde calea de date cu:
 - numărătorul **SCnt** (5 biți);
 - cu un **MUX**, care selectează între B și ALUOut.
- Se extinde unitatea de control cu semnalele **SCWr** și **SCEn**.
- Conexiuni necesare: $D \leftarrow sa$, $LD \leftarrow SCWr \ \& \ (opcode=0)$, $EN \leftarrow SCEn$ și $sa_{ALU} \leftarrow \overline{ZeroC}$.

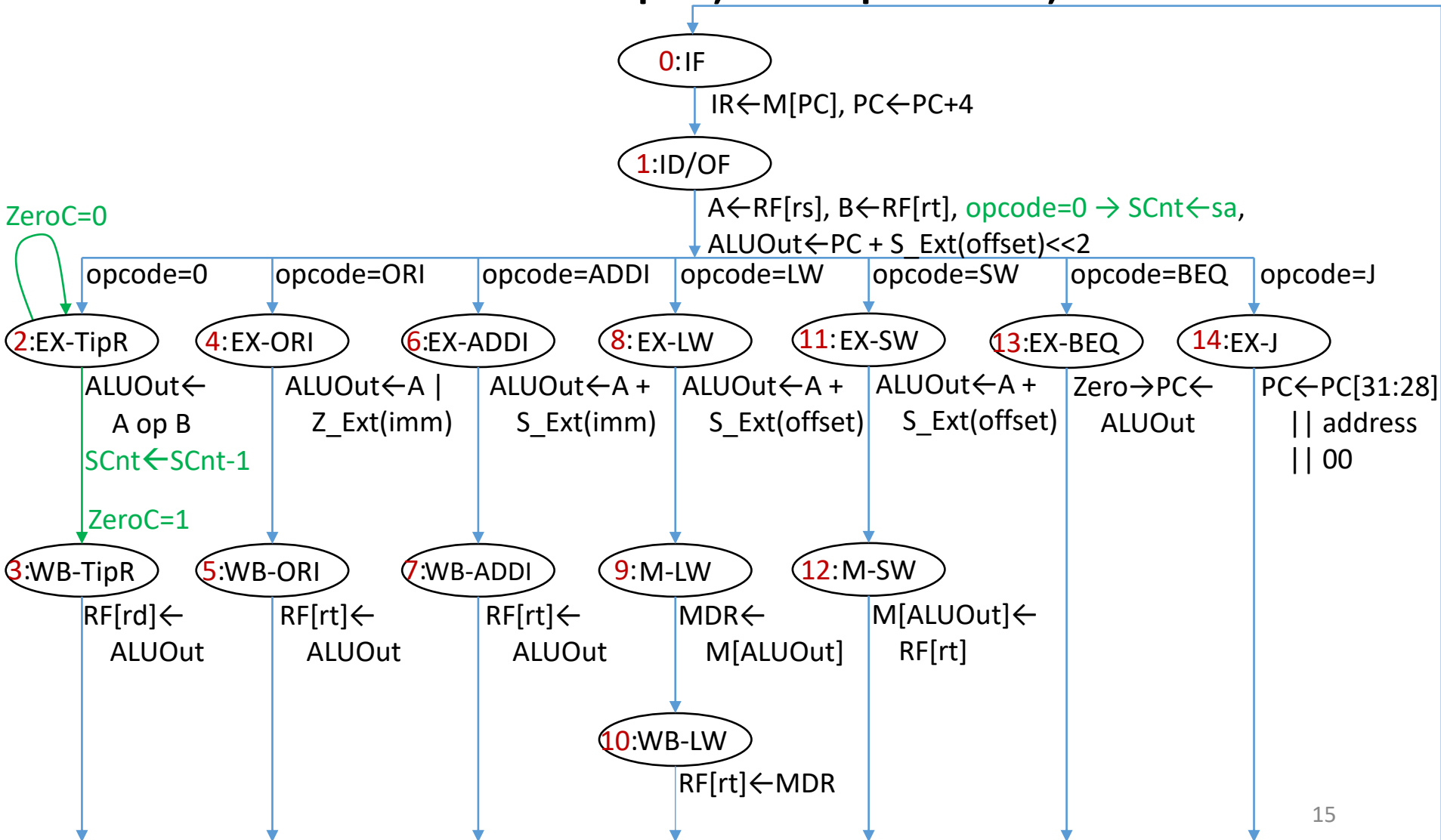
Notă: SCnt trebuie încărcat numai la TipR =>
 $LD \leftarrow SCWr \ \& \ (opcode=0)$



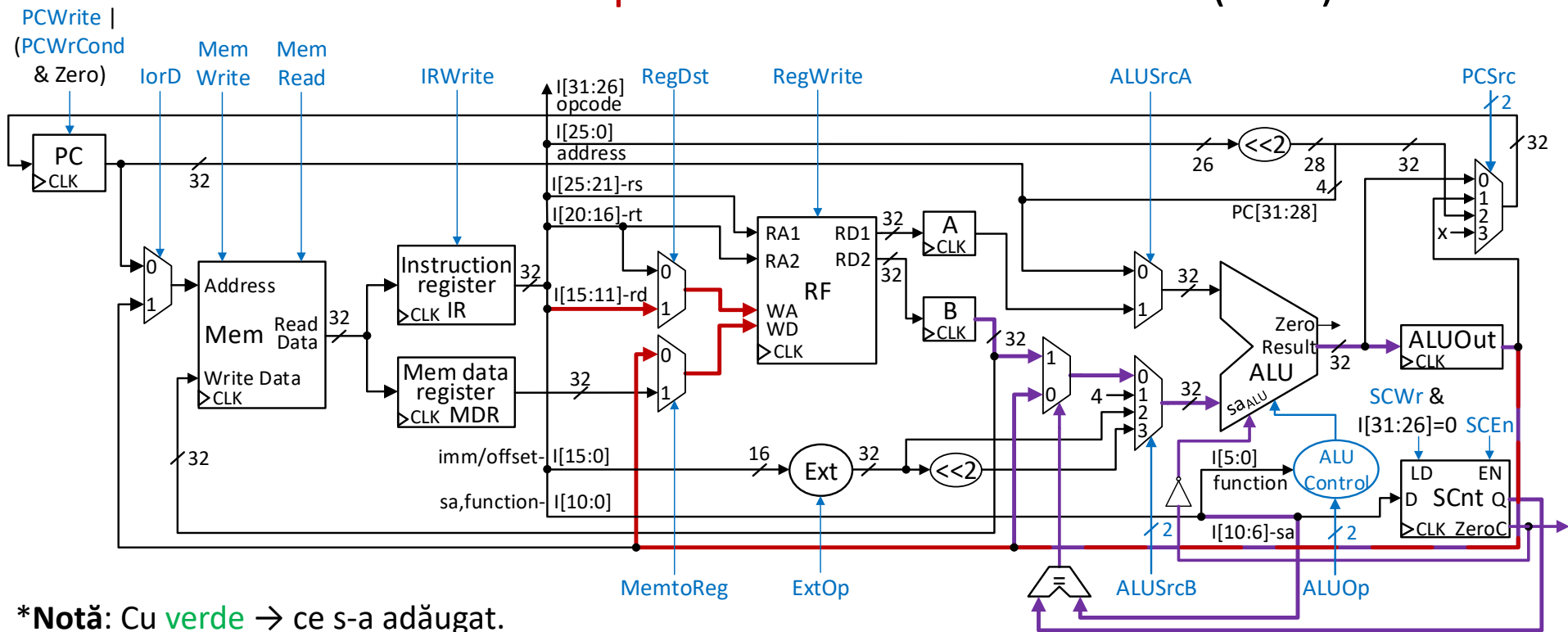
7.2.1 Implementarea cablată

7.2.1.4 Studiu de caz: Implementarea SLL multi-ciclu (cont)

FSM fără comasare cu repetiția EX-TipR în funcție de ZeroC



7.2.1.4 Studiu de caz: Implementarea SLL multi-ciclu (cont)



*Notă: Cu verde → ce s-a adăugat.

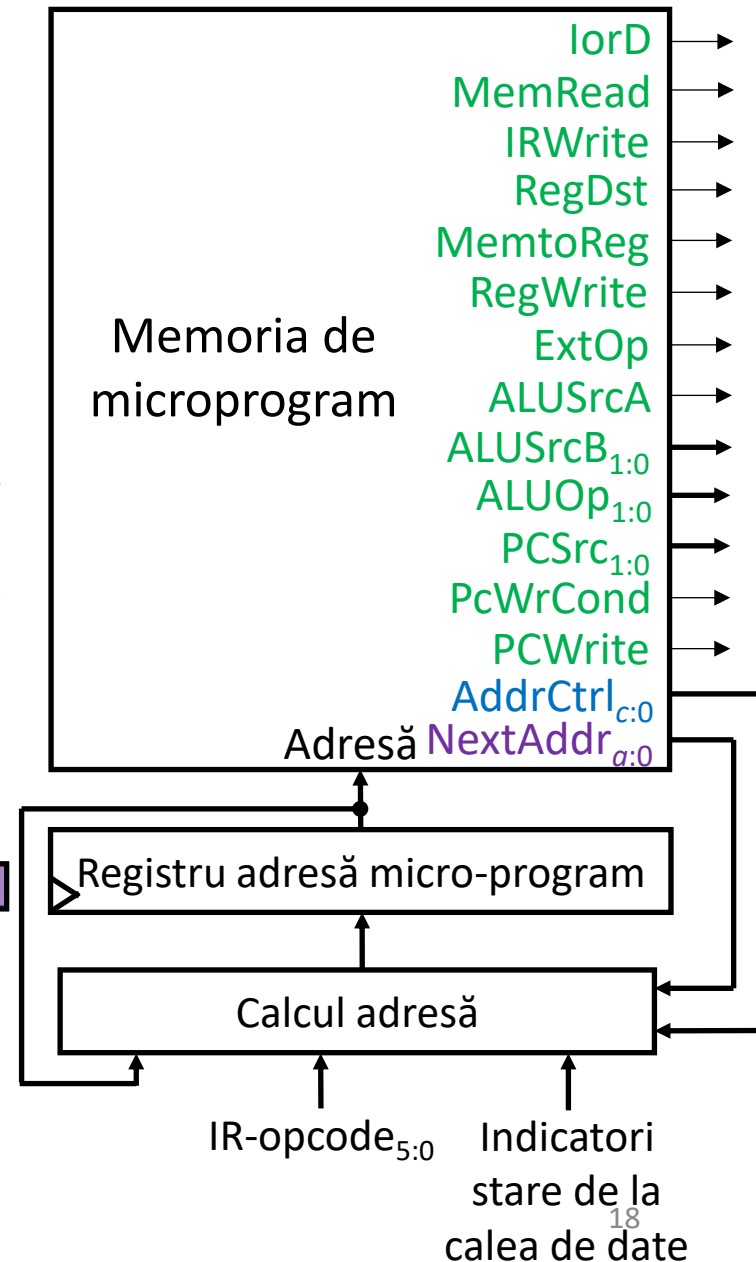
Pas	RTL concret	Semnale relevante
T_0	$IR \leftarrow M[PC], PC \leftarrow PC+4$	$\overline{IorD}$, $\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{SCWr}$, $\overline{SCEn}$, $\overline{ALUSrcA}$, $ALUSrcB=01$, $ALUOp=(+)$, $PCSrc=00$, $PCWrite$
T_1	$A \leftarrow R[rs], B \leftarrow R[rt], opcode=0 \rightarrow SCnt \leftarrow sa$, $ALUOut \leftarrow PC + S_Ext(offset) \ll 2$	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $ExtOp$, $\overline{SCWr}$, $\overline{SCEn}$, $\overline{ALUSrcA}$, $ALUSrcB=11$, $ALUOp=(+)$, $\overline{PCWrCond}$, $\overline{PCWrite}$
SLL: T_2	$\overline{SCnt}=sa \rightarrow ALUOut \leftarrow B \ll sa_{ALU}$, $\overline{SCnt} \neq sa \rightarrow ALUOut \leftarrow ALUOut \ll sa_{ALU}$, $\overline{SCnt} \leftarrow \overline{SCnt}-1$, $ZeroC=0 \rightarrow$ Repetă SLL: T_2	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $ALUSrcB=00$, $ALUOp=(R)$, $\overline{PCWrCond}$, $\overline{PCWrite}$, $\overline{SCWr}$, $\overline{SCEn}$
SLL: T_3	$R[rd] \leftarrow ALUOut$	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $RegDst$, $MemtoReg$, $\overline{RegWrite}$, $\overline{PCWrCond}$, $\overline{PCWrite}$, $\overline{SCWr}$, $\overline{SCEn}$

7.2.1 Generalități

- Micro-programare **pe orizontală**:



- stările sunt stocate ca micro-instrucțiuni în memorie la adresa dată de codul stării (0-14).
- micro-instrucțiunile conțin câte un bit pentru fiecare semnal de control => micro-instrucțiuni late și operații paralele multiple, care codifică direct tabelul semnalelor de control;
- codificare simplă.



7.2.2 Implementarea micro-programată

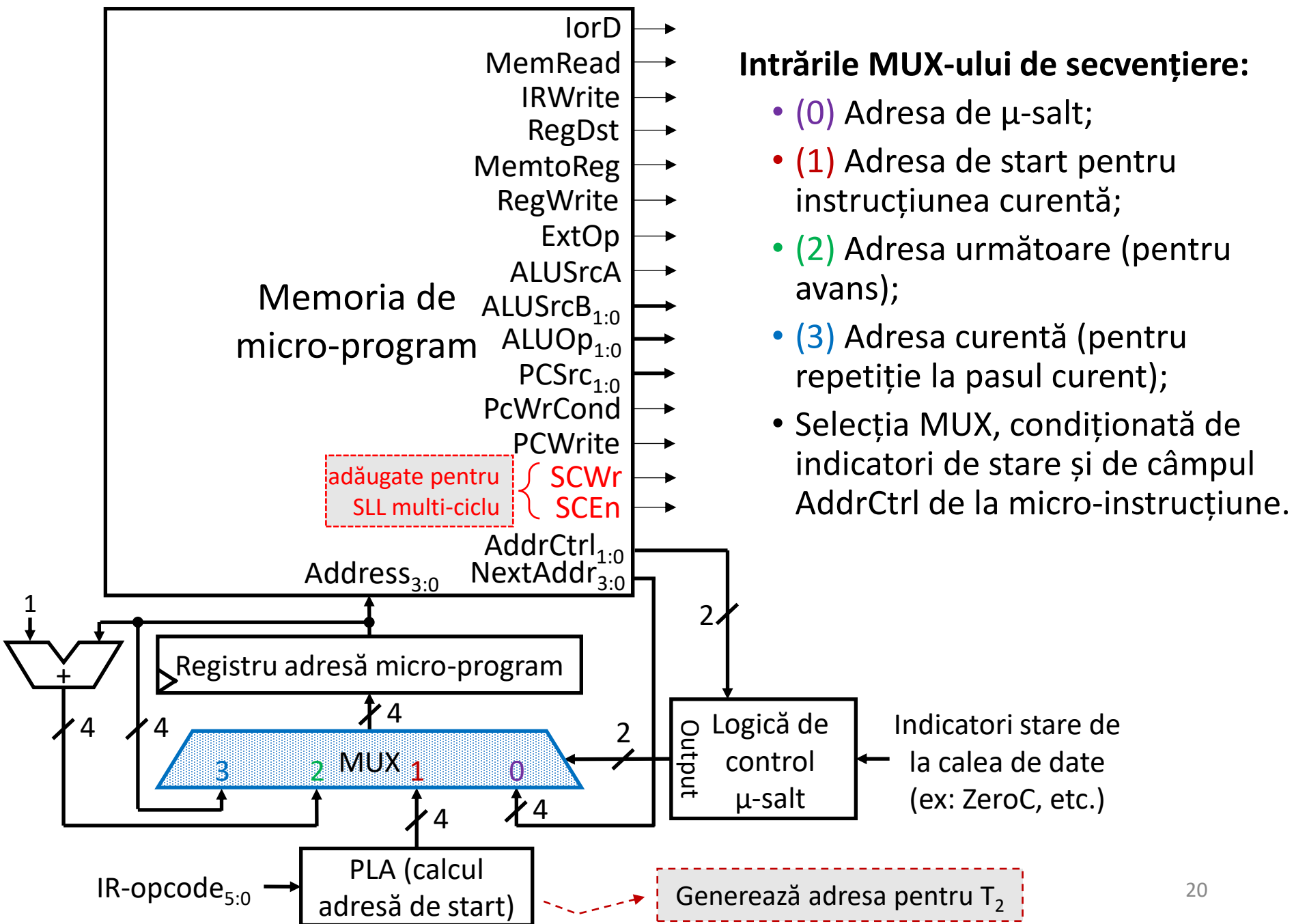
7.2.2 Caracteristici

- Simplu de proiectat.
- Flexibilitate – adaptare ușoară la schimbări de organizare și tehnologie;
– acceptă schimbări în faze avansate de proiectare sau ulterior.
- Scalabilitate – se pot implementa seturi de instrucțiuni complexe folosind memorii de micro-program mai mari.
- Generalitate – se pot implementa seturi de instrucțiuni pe aceeași arhitectură;
– se poate adapta setul de instrucțiuni în funcție de aplicație.
- Compatibilitate – acceptă organizări diferite pentru același set de instrucțiuni.
- Performanță – mai lent ca la implementarea cablată.

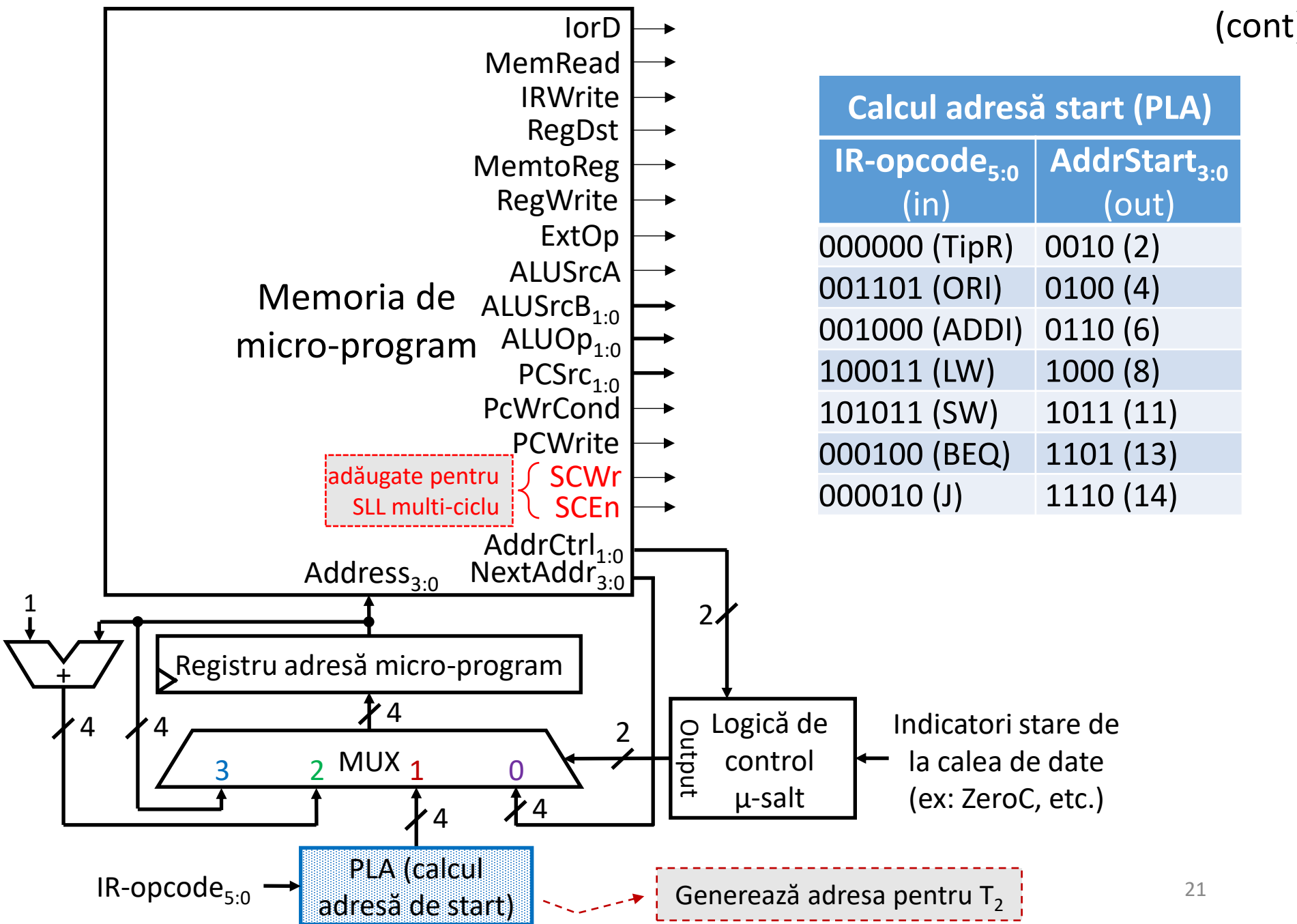
7.2.3 Proiectarea setului de micro-instrucțiuni

1. Se revizuieste lista semnalelor de control.
2. Se grupează semnalele de control în câmpuri (ex: semnale pentru ALU, memorie, PC, AddrCtrl, NextAddr, etc.).
3. Se ordonează câmpurile după semnificație (ex: operații ALU, memorare, control pentru selecția adresei următoare, adresa de micro-salt, etc.).
4. Se descrie la nivel simbolic formatul micro-instrucțiunilor specificând câmpurile și efectul lor asupra semnalelor de control.
5. Se aplică tehnici de minimizare a lățimii micro-instrucțiunilor.

Exemplu: Unitate Control cu micro-program orizontal și micro-salturi condiționate

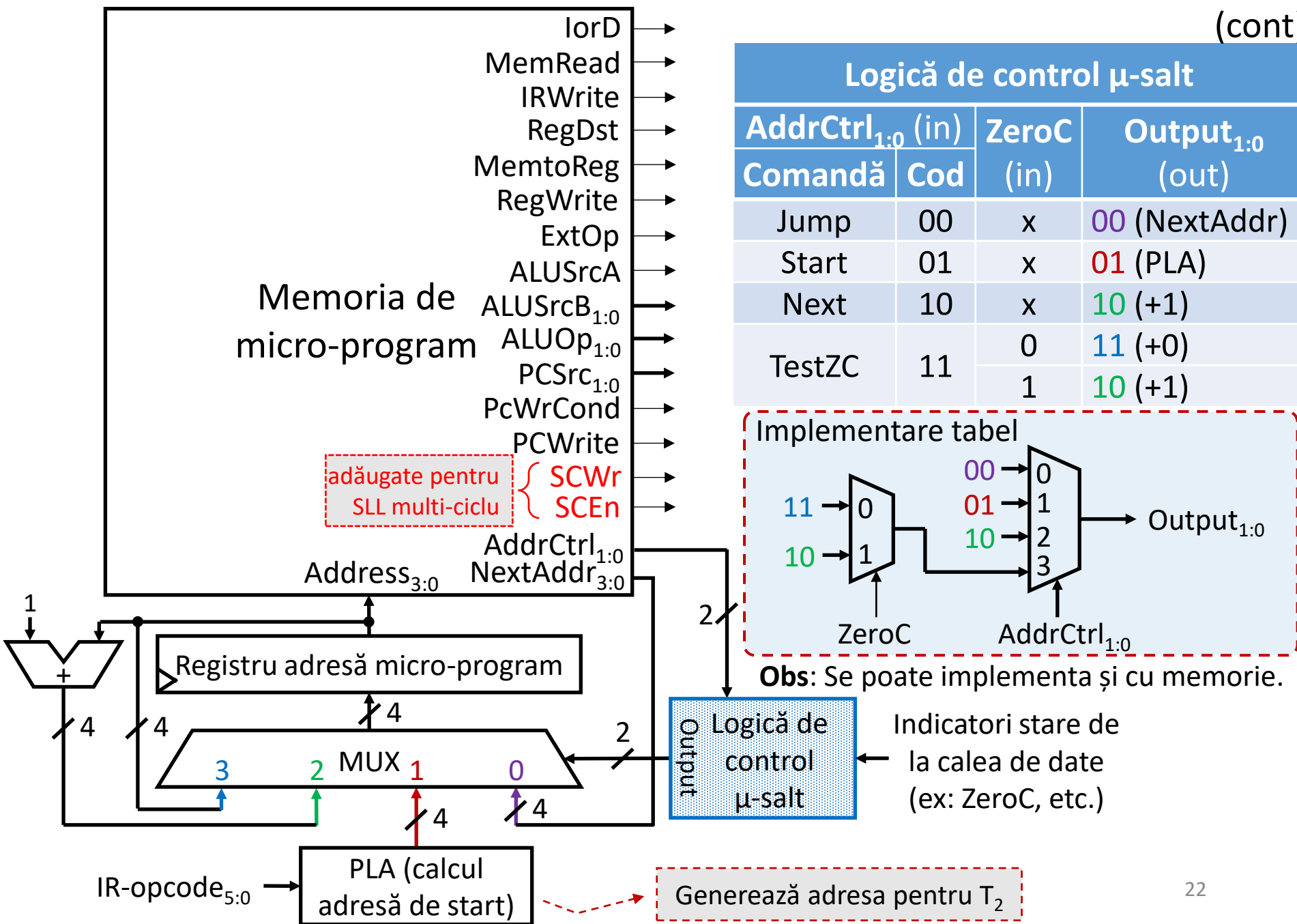


Exemplu: Unitate Control cu micro-program orizontal și micro-salturi condiționate (cont)

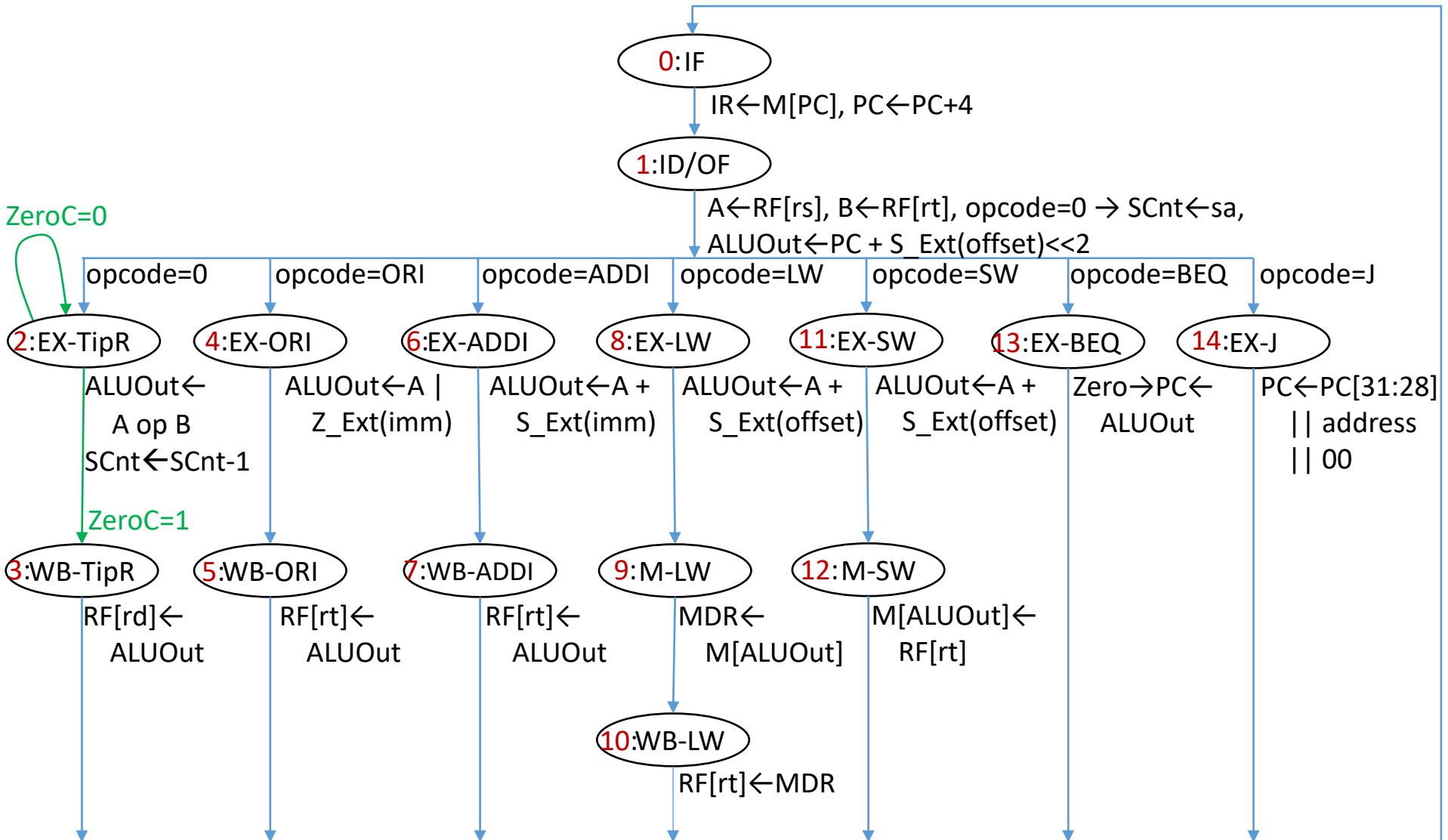


Calcul adresă start (PLA)	
IR-opcode _{5:0} (in)	AddrStart _{3:0} (out)
000000 (TipR)	0010 (2)
001101 (ORI)	0100 (4)
001000 (ADDI)	0110 (6)
100011 (LW)	1000 (8)
101011 (SW)	1011 (11)
000100 (BEQ)	1101 (13)
000010 (J)	1110 (14)

Exemplu: Unitate Control cu micro-program orizontal și micro-salturi condiționate (cont)



Exemplu: Unitate Control cu micro-program orizontal și micro-salturi condiționate – FSM fără comasare cu repetiția EX-TipR în funcție de ZeroC – (cont)



Exemplu: Unitate Control cu micro-program orizontal și micro-salturi condiționate – Memoria de micro-program – (cont)

Adr	IorD	Mem Read	Mem Write	IR Write	Reg Dst	Mem toReg	Reg Wr	Ext Op	ALU SrcA	ALU SrcB	ALU Op	PC Src	PCW rCnd	PC Wr	SC En	SC Wr	Addr Ctrl	Next Addr	Fază execuție
–	1b	1b	1b	1b	1b	1b	1b	1b	1b	2b	2b	2b	1b	1b	1b	1b	2b	4b	–
0	0	1	0	1	x	x	0	x	0	01	00 (+)	00	x	1	0	0	10 (Next)	xxxx	IF
1	x	0	0	0	x	x	0	1	0	11	00 (+)	xx	0	0	0	1	01 (Start)	xxxx	ID, OF
2	x	0	0	0	x	x	0	x	1	00	10 (R)	xx	0	0	1	0	11 (TestZC)	xxxx	EX-TipR
3	x	0	0	0	1	0	1	x	x	xx	xx	xx	0	0	0	0	00 (Jump)	0000	WB-TipR
4	x	0	0	0	x	x	0	0	1	10	11 ()	xx	0	0	0	0	10 (Next)	xxxx	EX-ORI
5	x	0	0	0	0	0	1	x	x	xx	xx	xx	0	0	0	0	00 (Jump)	0000	WB-ORI
6	x	0	0	0	x	x	0	1	1	10	00 (+)	xx	0	0	0	0	10 (Next)	xxxx	EX-ADDI
7	x	0	0	0	0	0	1	x	x	xx	xx	xx	0	0	0	0	00 (Jump)	0000	WB-ADDI
8	x	0	0	0	x	x	0	1	1	10	00 (+)	xx	0	0	0	0	10 (Next)	xxxx	EX-LW
9	1	1	0	0	x	x	0	x	x	xx	xx	xx	0	0	0	0	10 (Next)	xxxx	M-LW
10	x	0	0	0	0	1	1	x	x	xx	xx	xx	0	0	0	0	00 (Jump)	0000	WB-LW
11	x	0	0	0	x	x	0	1	1	10	00 (+)	xx	0	0	0	0	10 (Next)	xxxx	EX-SW
12	1	0	1	0	x	x	0	x	x	xx	xx	xx	0	0	0	0	00 (Jump)	0000	M-SW
13	X	0	0	0	x	x	0	x	1	00	01 (-)	01	1	0	0	0	00 (Jump)	0000	EX-BEQ
14	x	0	0	0	x	x	0	x	x	xx	xx	10	x	1	0	0	00 (Jump)	0000	EX-J

Micro-program pe orizontală de dimensiune 15x25 biți.

7.3 Mecanismul de tratare a excepțiilor

Definiție: Excepția = eveniment care întrerupe fluxul de execuție și îl transferă la rutine speciale de tratare a acestuia.

Tipuri:

- Excepție la depășire (la calcule aritmetice cu semn) – se generează în procesor.
- Întrerupere (de intrare/ieșire) – se generează din cauze externe (de la periferice I/O).

Detectarea excepției

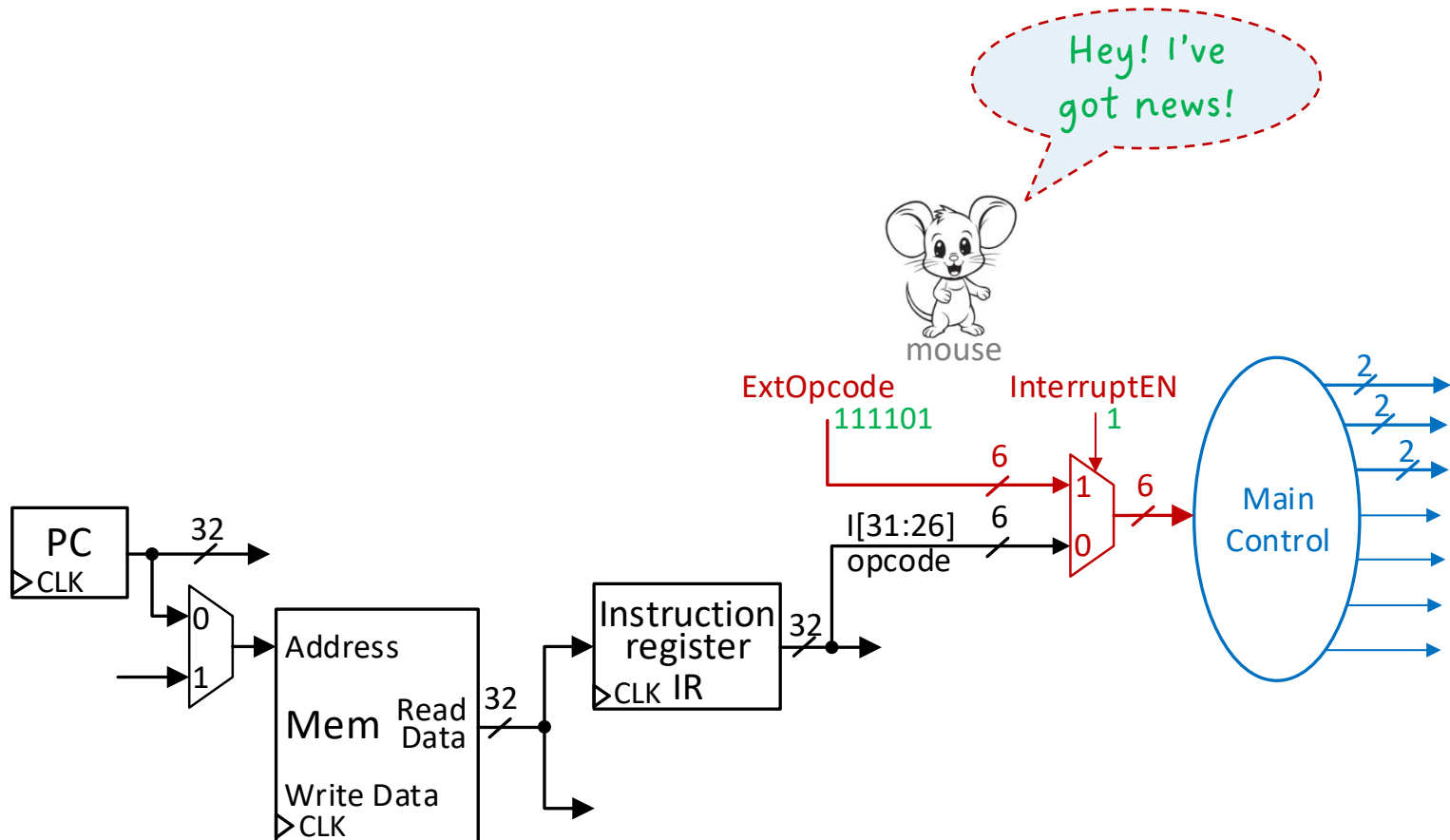
- La **depășire** – pe baza indicatorului **Ovr** (overflow) de la ALU (vezi curs 5), care va determina saltul în starea **Overflow** introdusă în FSM.
- La **întrerupere** – apare un *opcode* nedefinit, care va determina saltul în starea **IllegalOp** introdusă în FSM.

Tratarea excepției

1. Registrul **CP₀:EPC** (Exception Program Counter) memorează adresa instrucțiunii curente:
$$\text{EPC} \leftarrow \text{PC} - 4 \text{ (deoarece } \text{PC} = \text{PC} + 4 \text{ după IF).}$$
2. Registrul **CP₀:Cause** se scrie cu codul asociat cauzei întreruperii:
 - La depășire: $\text{Cause} \leftarrow 1$
 - La întrerupere: $\text{Cause} \leftarrow 0$
3. Se face salt la rutina de tratare a excepțiilor:
$$\text{PC} \leftarrow \text{x80000180}$$
4. În rutina de tratare a excepțiilor se acționează conform cauzei salvate în **CP₀:Cause**, iar la final se face salt (revenire) la instrucțiunea indicată de **CP₀:EPC** sau **CP₀:EPC+4**.²⁵

7.3 Mecanismul de tratare a excepțiilor

Q: Cum/Unde apare un *opcode* nedefinit din exterior, la întrerupere?

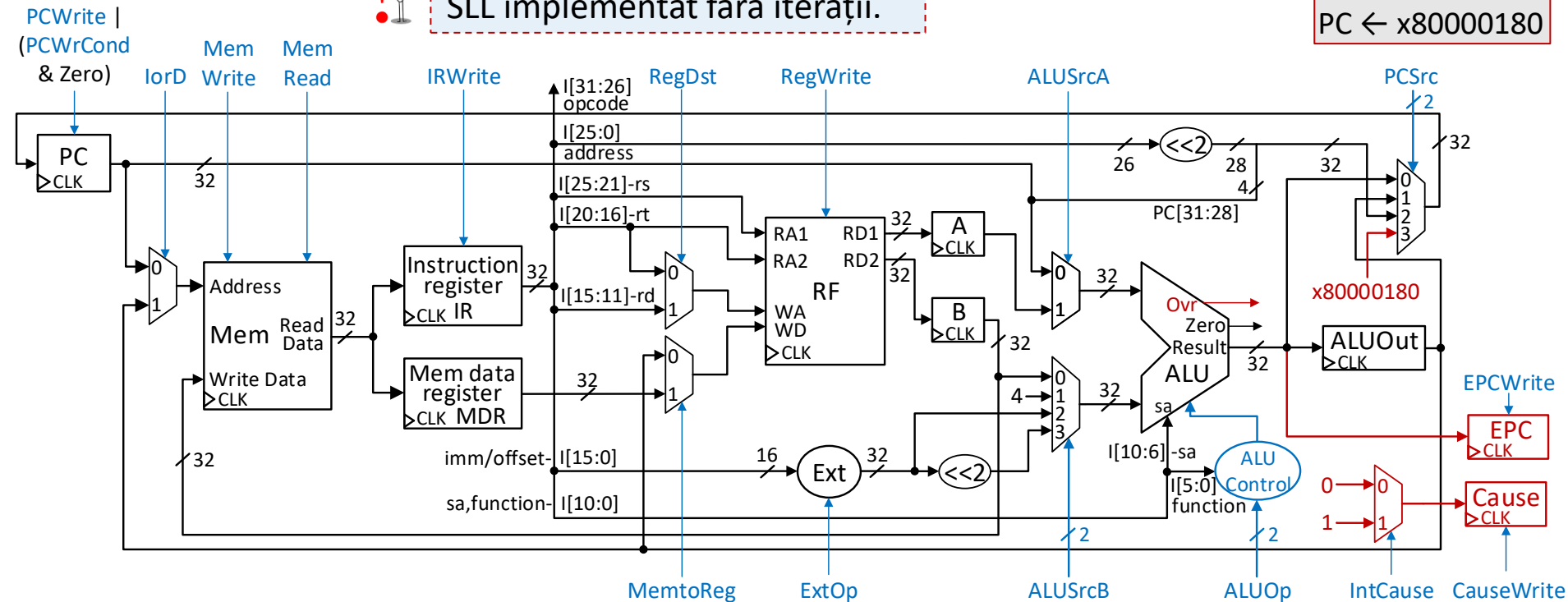


7.3.1 Extinderea procesorului



Notă: În rezolvare considerăm SLL implementat fără iterații.

```
EPC ← PC - 4
Cause ← 0 sau 1
PC ← x80000180
```

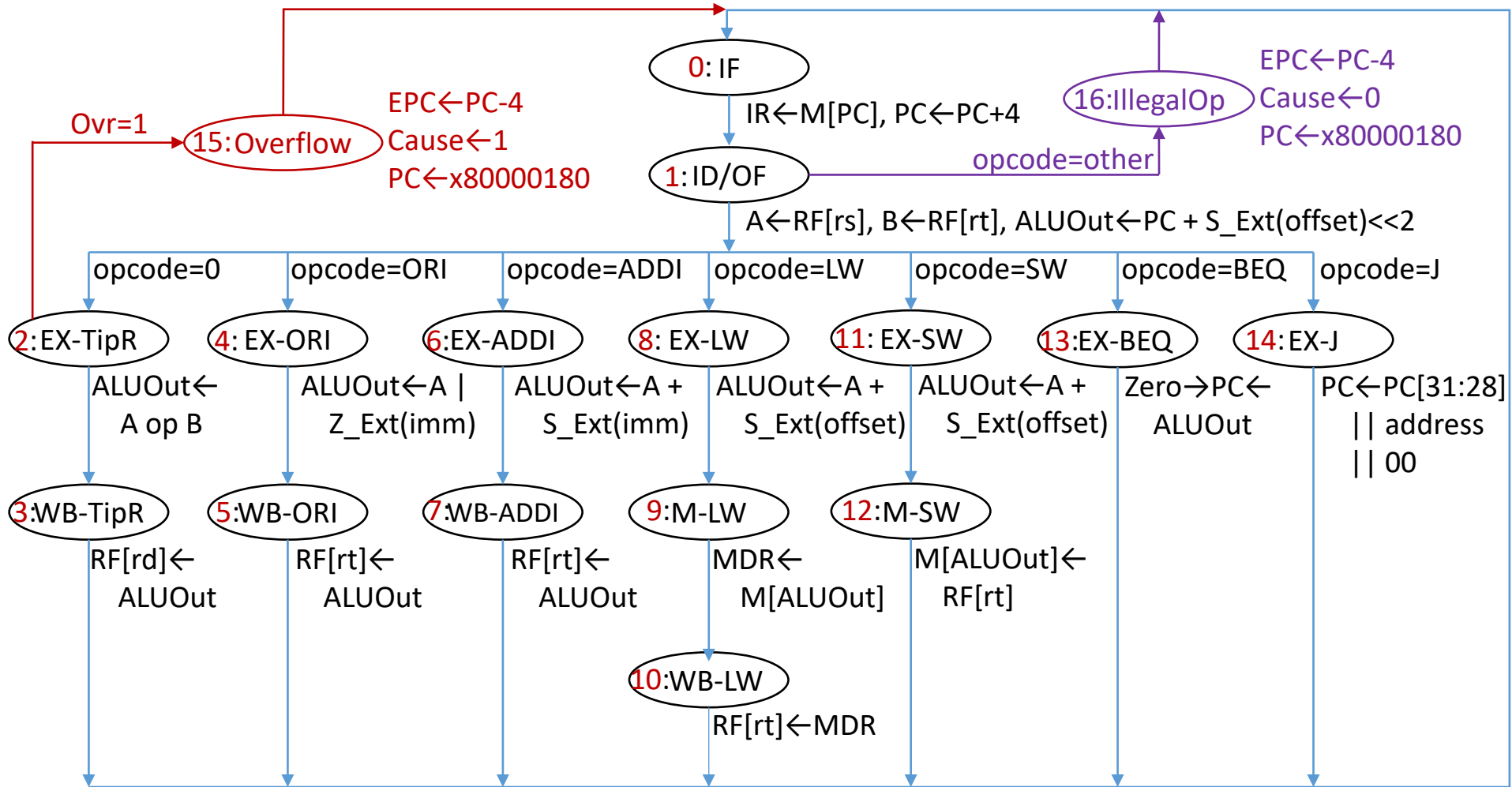


Elemente adăugate

- Registre noi: **EPC**, **Cause** => semnale suplimentare de validare a scrierii: **EPCWrite**, **CauseWrite**.
 - Multiplexor pentru valoarea registrului **Cause** cu selecția **IntCause** (1 – depășire; 0 – întrerupere).
 - Intrare nouă la MUX cu selecția **PCSrc**: (**PCSrc**=3) → PC ← x80000180.
 - Indicator de overflow **Ovr** generat de ALU.
- 27

7.3 Mecanismul de tratare a excepțiilor

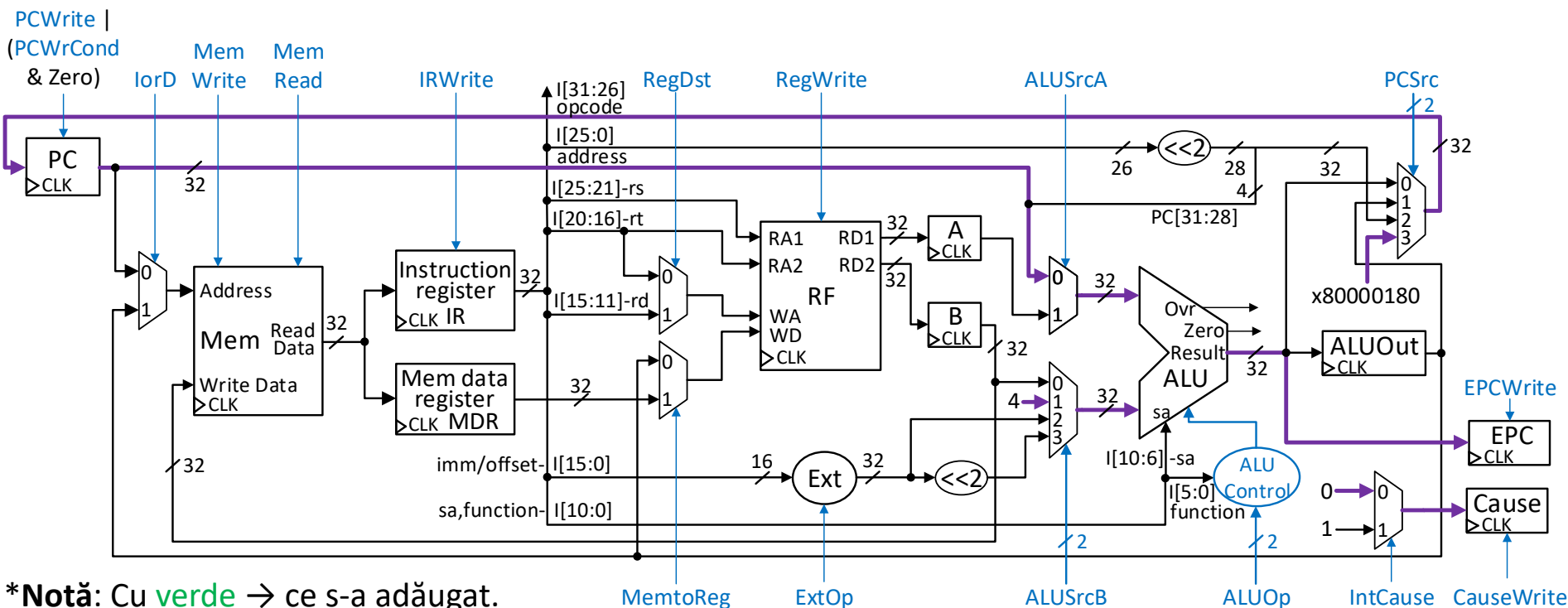
7.3.2 Extinderea FSM



Elemente adăugate:

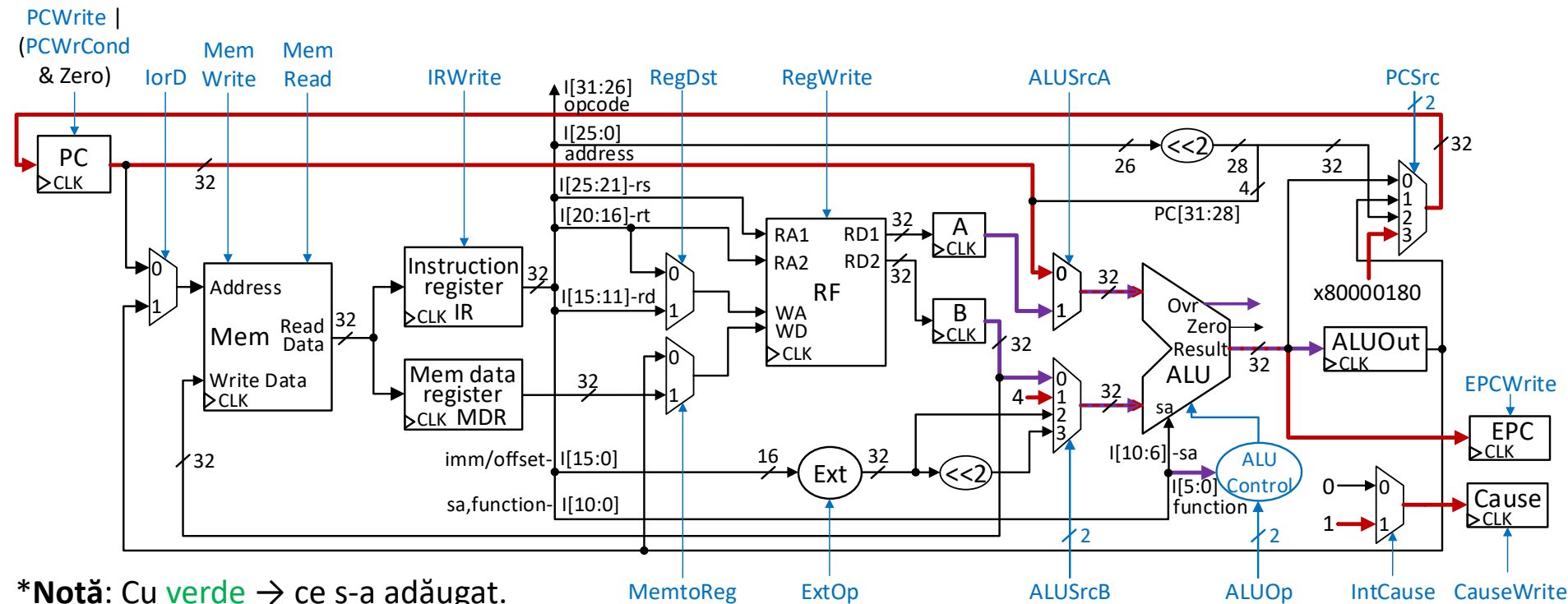
- 2 stări **Overflow** și **IllegalOp**, numerotate **15**, respectiv **16** – sunt **necesari 5 biți** la codificare.
- Tranziții și RTL concret pentru stările adăugate. Scăderea PC-4 se va face în ALU.

7.3.3 Excepția de tip *întrerupere* – RTL și semnale de control



Pas	RTL concret	Semnale relevante
T_0	$IR \leftarrow M[PC], PC \leftarrow PC + 4$	$\overline{IorD}$, $\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB=01}$, $\overline{ALUOp=(+)}$, $\overline{PCSrc=00}$, $\overline{PCWrite}$, CauseWrite , EPCWrite
T_1	$A \leftarrow R[rs], B \leftarrow R[rt]$, $ALUOut \leftarrow PC + S_Ext(offset) \ll 2$ (Opcode=other) → salt la IllegalOp	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ExtOp}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB=11}$, $\overline{ALUOp=(+)}$, $\overline{PCWrCond}$, $\overline{PCWrite}$, CauseWrite , EPCWrite
Illegal Op: T_2	$EPC \leftarrow PC - 4$, $Cause \leftarrow 0$, $PC \leftarrow x80000180$	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB=01}$, $\overline{ALUOp=(-)}$, $\overline{PCSrc=11}$, $\overline{PCWrite}$, $\overline{IntCause}$, $\overline{CauseWrite}$, EPCWrite

7.3.4 Excepția de tip *overflow* – RTL și semnale de control



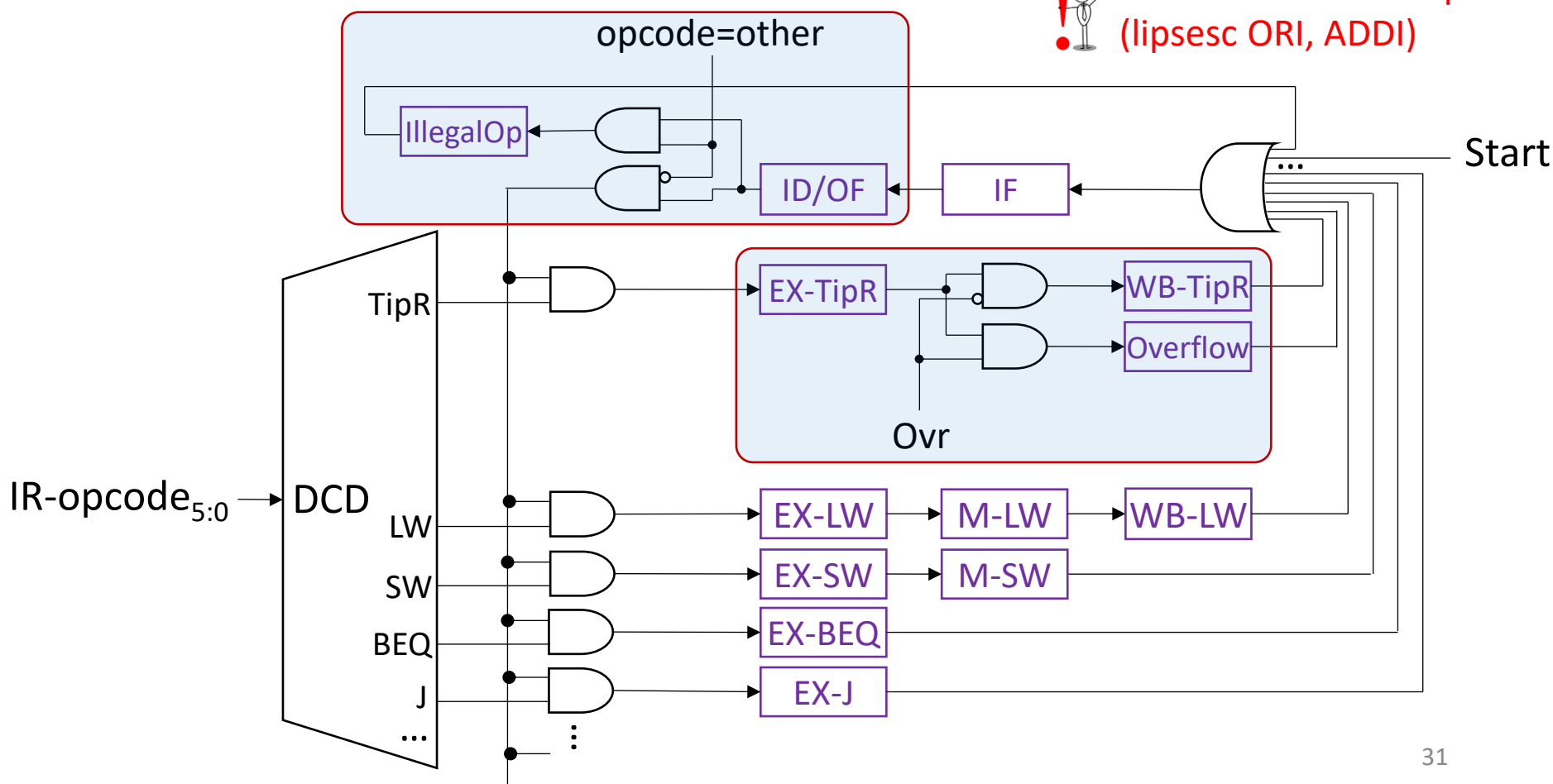
Pas	RTL concret	Semnale relevante
T_0	$IR \leftarrow M[PC], PC \leftarrow PC+4$	$\overline{IorD}$, $\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB}=01$, $\overline{ALUOp}=(+)$, $\overline{PCSrc}=00$, $\overline{PCWrite}$, <u>CauseWrite</u> , <u>EPCWrite</u>
T_1	$A \leftarrow R[rs], B \leftarrow R[rt]$ $ALUOut \leftarrow PC + S_Ext(offset) \ll 2$	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ExtOp}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB}=11$, $\overline{ALUOp}=(+)$, $\overline{PCWrCond}$, $\overline{PCWrite}$, <u>CauseWrite</u> , <u>EPCWrite</u>
TipR: T_2	$ALUOut \leftarrow A \text{ op } B$, Ovr → Salt la Overflow: T_3	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB}=00$, $\overline{ALUOp}=(R)$, $\overline{PCWrCond}$, $\overline{PCWrite}$, <u>CauseWrite</u> , <u>EPCWrite</u>
Over flow: T_3	$EPC \leftarrow PC-4$, $Cause \leftarrow 1$, $PC \leftarrow x80000180$	$\overline{MemRead}$, $\overline{MemWrite}$, $\overline{IRWrite}$, $\overline{RegWrite}$, $\overline{ALUSrcA}$, $\overline{ALUSrcB}=01$, $\overline{ALUOp}=(-)$, $\overline{PCSrc}=11$, $\overline{PCWrite}$, $\overline{IntCause}$, $\overline{CauseWrite}$, <u>EPCWrite</u>

7.3.5 Excepții: Implementarea cablată cu un bistabil pe stare

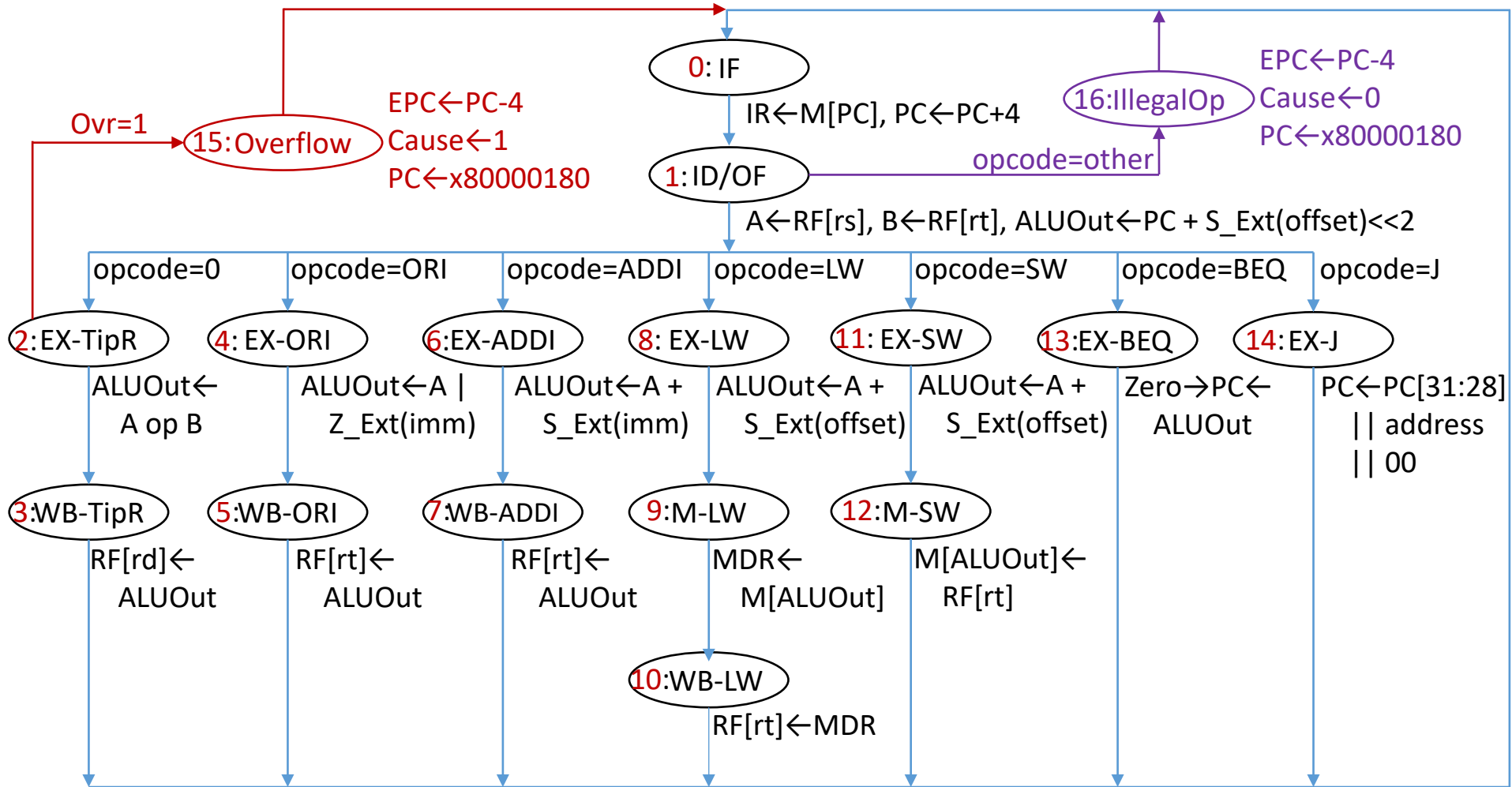
- Tranziții suplimentare:

- **Opcode=other** va determina activarea bistabilului **IllegalOp** după bistabilul **IF**.
- **Ovr=1** va determina activarea bistabilului **Overflow** după bistabilul **EX-TipR**.

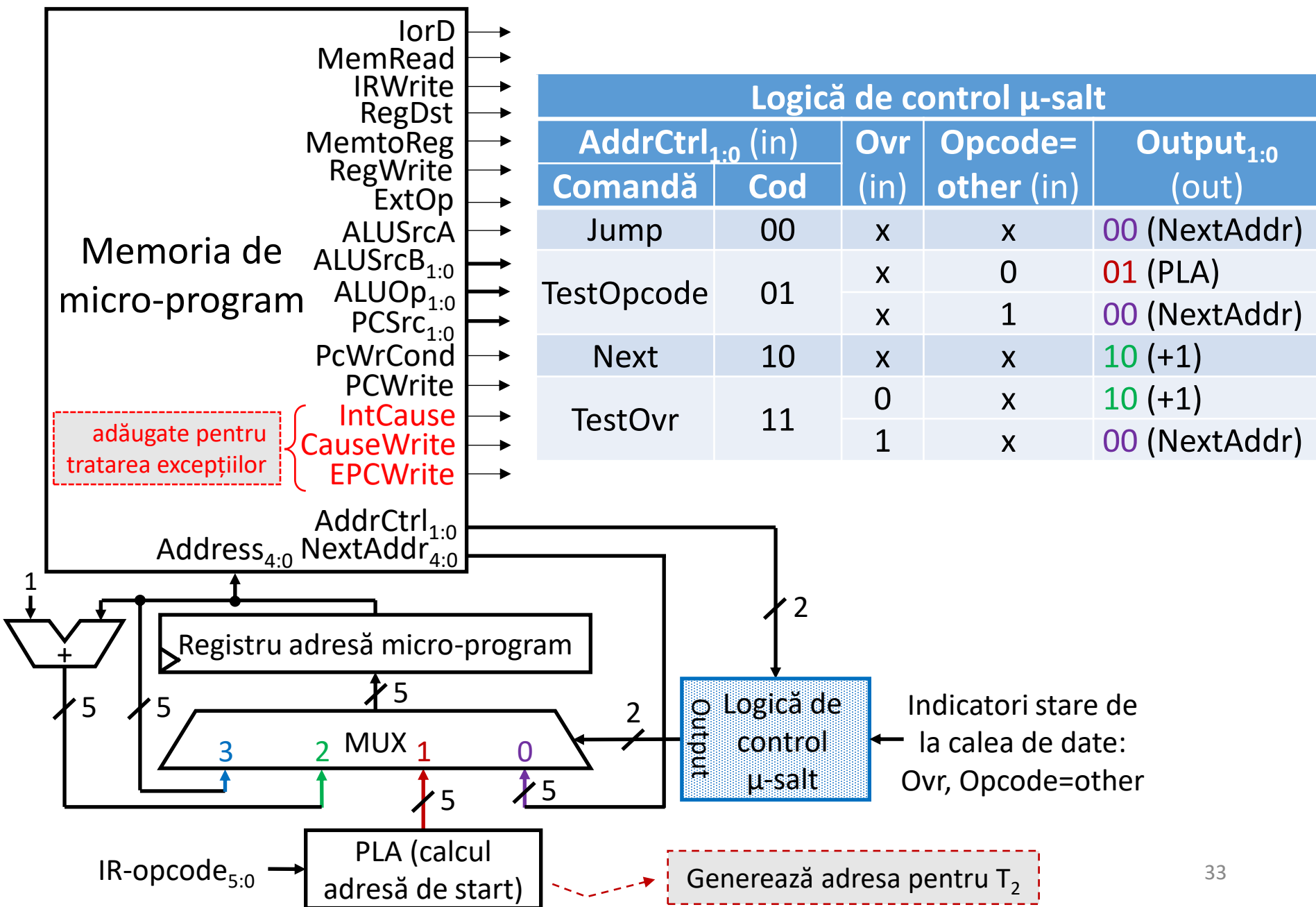
!Schema este incompletă!
(lipsește ORI, ADDI)



7.3.6.1 Excepții: Implementarea **micro-programată** – **FSM** (ca la implementarea cu un bistabil pe stare)



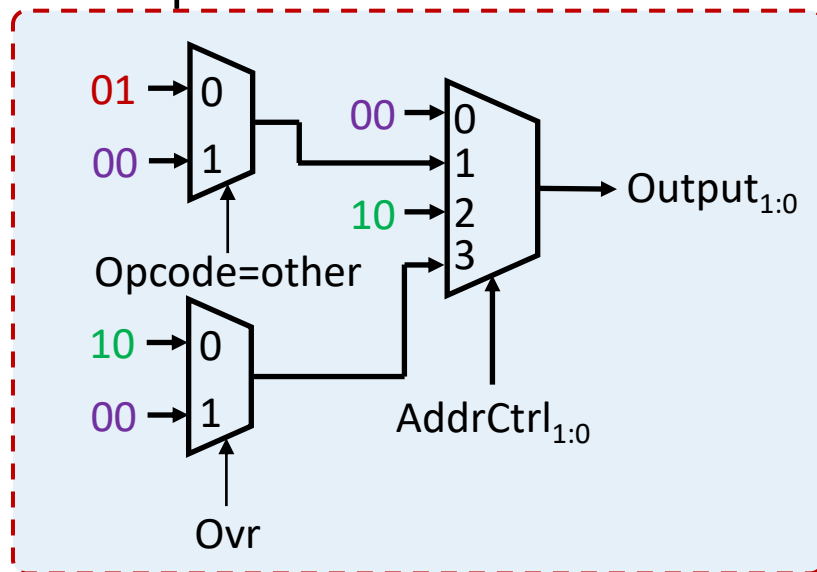
7.3.6.2 Excepții: Implementarea **micro-programată** – Arhitectura



7.3.6.3 Excepții: Implementarea Logicii de Control μ -salt

Logică de control μ -salt				
AddrCtrl _{1:0} (in)		Ovr (in)	Opcode=other (in)	Output _{1:0} (out)
Comandă	Cod			
Jump	00	x	x	00 (NextAddr)
TestOpcode	01	x	0	01 (PLA)
		x	1	00 (NextAddr)
Next	10	x	x	10 (+1)
TestOvr	11	0	x	10 (+1)
		1	x	00 (NextAddr)

Implementare cu MUX-uri



Obs: Se poate implementa și cu memorie.

7.3.6.4 Excepții: Memoria de micro-program

Adr	lor D	Mem Read	Mem Write	IR Write	Reg Dst	Mem toReg	Reg Wr	Ext Op	ALU SrcA	ALU SrcB	ALU Op	PC Src	PCW rCnd	PC Wr	IntC ause	Caus eWr	EPC Wr	AddrCtrl	Next Addr	Fază execuție
–	1b	1b	1b	1b	1b	1b	1b	1b	1b	2b	2b	2b	1b	1b	1b	1b	1b	2b	5b	–
0	0	1	0	1	x	x	0	x	0	01	00(+)	00	x	1	x	0	0	10(Next)	xxxxx	IF
1	x	0	0	0	x	x	0	1	0	11	00(+)	xx	0	0	x	0	0	01(TstOpcode)	10000	ID, OF
2	x	0	0	0	x	x	0	x	1	00	10(R)	xx	0	0	x	0	0	11(TstOvrflow)	01111	EX-TipR
3	x	0	0	0	1	0	1	x	x	xx	xx	xx	0	0	x	0	0	00(Jump)	00000	WB-TipR
4	x	0	0	0	x	x	0	0	1	10	11()	xx	0	0	x	0	0	10(Next)	xxxxx	EX-ORI
5	x	0	0	0	0	0	1	x	x	xx	xx	xx	0	0	x	0	0	00(Jump)	00000	WB-ORI
6	x	0	0	0	x	x	0	1	1	10	00(+)	xx	0	0	x	0	0	10(Next)	xxxxx	EX-ADDI
7	x	0	0	0	0	0	1	x	x	xx	xx	xx	0	0	x	0	0	00(Jump)	00000	WB-ADDI
8	x	0	0	0	x	x	0	1	1	10	00(+)	xx	0	0	x	0	0	10(Next)	xxxxx	EX-LW
9	1	1	0	0	x	x	0	x	x	xx	xx	xx	0	0	x	0	0	10(Next)	xxxxx	M-LW
10	x	0	0	0	0	1	1	x	x	xx	xx	xx	0	0	x	0	0	00(Jump)	00000	WB-LW
11	x	0	0	0	x	x	0	1	1	10	00(+)	xx	0	0	x	0	0	10(Next)	xxxxx	EX-SW
12	1	0	1	0	x	x	0	x	x	xx	xx	xx	0	0	x	0	0	00(Jump)	00000	M-SW
13	x	0	0	0	x	x	0	x	1	00	01(-)	01	1	0	x	0	0	00(Jump)	00000	EX-BEQ
14	x	0	0	0	x	x	0	x	x	xx	xx	10	x	1	x	0	0	00(Jump)	00000	EX-J
15	x	0	0	0	x	x	0	x	0	01	01(-)	11	x	1	1	1	1	00(Jump)	00000	Overflow
16	x	0	0	0	x	x	0	x	0	01	01(-)	11	x	1	0	1	1	00(Jump)	00000	IllegalOp

Micro-program pe orizontală de dimensiune 17x27 biți.

Probleme propuse

Implementarea altor instrucțiuni (vezi Curs 4, la final – Probleme propuse) și proiectarea modificărilor în unitatea de control (cablat/microprogramat) pentru a suporta noile instrucțiuni.

Referințe

- [1] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, **3rd edition**, Cap. 5.5, 5.6, Ed. Morgan–Kaufmann, 2004.
- [2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Anexa D, Ed. Morgan–Kaufmann, 2013.
- [3] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [4] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.